

CLASS DESIGN API's INTERACTIONS

APPLICATION NAME: VIDYA SAATHI

DOMAIN: EDTECH MOBILE WEB | GENERATIVE AI

TEAM DETAILS: TEAM VIDYASETU

Prepared by:

- Manideep Thalluru
- Janani TR
- Charan Sathwik Kolapudi
- Rejeeth Sarvan M

1.1 Introduction

The Class Design represents the object-oriented architecture of Vidya Saathi—an AI-Powered Personalized Learning Companion for Students. It provides a comprehensive structural view of the system software by identifying the primary classes, their constituent attributes, encapsulate operations, and the physical relationships among them. Each class is engineered to execute a single decoupled responsibility, ensuring that the Phase 1 Minimum Viable Product (MVP) workspace remains highly modular, cohesive, and easily maintainable. This architectural blueprint translates the frozen functional specifications into solid object-oriented source components, guiding the engineering team directly into the codebase build implementation.

The primary scope of this class design covers the encapsulation of core student flows including secure local identity profiling, Google OAuth credential resolution, OTP verification tracking, automated language-specific doubt management, dynamically adjusted MCQ quiz evaluation pipelines, and lookup logic rules defining the adaptive learning progression paths.

1.2 Design Objectives

The objectives of the class design are:

- To divide the system into manageable and reusable classes.
- To improve code maintainability and readability.
- To promote high cohesion and low coupling between classes.
- To support code reusability.
- To make future enhancements easier.
- To provide a clear representation of system functionality.
- To improve software scalability and flexibility.

1.3 Design Principles

The class design follows the following principles:

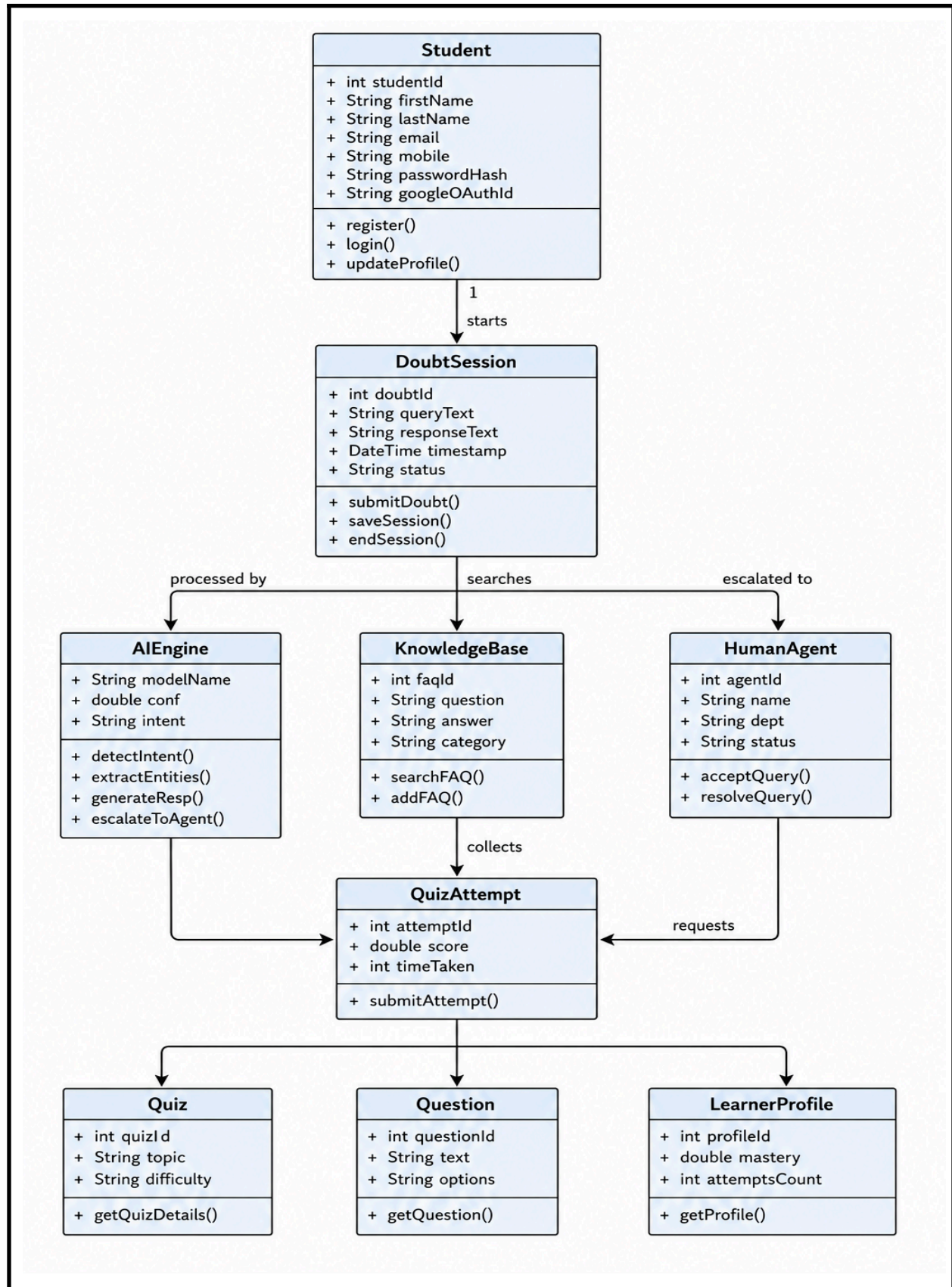
- **Encapsulation:** Each class stores its own data and methods.
- **Abstraction:** Only essential operations are exposed.
- **Inheritance:** Future educational services can inherit common properties.
- **Polymorphism:** Different response handlers can process different localized language queries.
- **Low Coupling:** Classes communicate through interfaces.
- **High Cohesion:** Each class performs only one responsibility.

1.4 Visibility of Members

The standard visibility indicators utilized within the class components are defined as follows:

Symbol	Visibility
+	Public
-	Private
#	Protected

1.5 UML Diagram



1.6 Class Description

Student

Purpose: Manages student registration, login, and profile updates.

Category	Details
Attributes	studentId, firstName, lastName, email, mobile, passwordHash, googleOAuthId
Methods	register(), login(), updateProfile()

Conversation (DoubtSession)

Purpose: Stores chat conversations between the student and the AI companion.

Category	Details
Attributes	doubtId, queryText, responseText, timestamp, status
Methods	submitDoubt(), saveSession(), endSession()

AIEngine

Purpose: Processes student queries and generates AI responses.

Category	Details
Attributes	modelName, confidence, intent
Methods	detectIntent(), extractEntities(), generateResponse(), escalateToAgent()

KnowledgeBase

Purpose: Stores educational FAQs and knowledge information.

Category	Details
Attributes	faqId, question, answer, category
Methods	searchFAQ(), addFAQ(), updateFAQ()

HumanAgent

Purpose: Handles queries transferred by the AI.

Category	Details
Attributes	agentId, name, department, status
Methods	acceptQuery(), resolveQuery()

QuizAttempt

Purpose: Collects data scores and results achieved during testing sessions.

Category	Details
Attributes	attemptId, score, timeTaken
Methods	submitAttempt()

Quiz

Purpose: Stores specific generated quiz evaluation settings.

Category	Details
Attributes	quizId, topic, difficulty
Methods	getQuizDetails()

Question

Purpose: Stores individual test item entries.

Category	Details
Attributes	questionId, text, options
Methods	getQuestion()

LearnerProfile

Purpose: Tracks student progress benchmarks over time.

Category	Details
Attributes	profileId, mastery, attemptsCount
Methods	getProfile()

1.7 Relationship Between Classes

The primary relationships among the classes exist as follows:

Relationship	Description
Student → DoubtSession	One student can have multiple active doubt tracking conversations.
DoubtSession → AIEngine	The AI Engine processes student queries to build textual solutions.
AIEngine → KnowledgeBase	Searches FAQs and course rules to resolve known concept entries.
DoubtSession → HumanAgent	Complex doubts or low-confidence interactions scale to human experts.
AIEngine → QuizAttempt	Directly initiates assessment checkpoints based on processed lesson context.
QuizAttempt → Quiz	Compiles specific questionnaire sheets matching chosen difficulty conditions.
QuizAttempt → Question	Evaluates individual parameters to trace right and wrong response lines.

QuizAttempt → LearnerProfile	Saves assessment outcomes to adjust dynamic systemic proficiency values.
Student → QuizAttempt	Tracks diagnostic scores compiled by individual accounts during evaluation rounds.

Section 2: External API Specifications

All external integrations are invoked exclusively from the **Application Layer** (never directly from the mobile/web client frontend). This preserves security by keeping authentication tokens hidden on the server and ensures structural consistency with the High-Level Technical Architecture.

2.1 Core API Integration Summary

External System	Endpoint / Protocol	Primary Purpose	Key Data Handled
Generative Language Model API (e.g., Google Gemini / OpenAI)	HTTPS POST/v1/generate	Generates localized doubt explanations and handles dynamic MCQ quiz authoring.	question_text, language, topic_id, difficulty_level
Google OAuth 2.0 Identity API	HTTPS POST & GET/oauth2/v3/userinfo	Authenticates or registers users via the mobile "Continue with Google" button.	id_token, googleOAuthId, email
SMS Gateway API (e.g., Twilio / Fast2SMS)	HTTPS POST/v1/otp/send & /verify	Dispatches and verifies 6-digit registration OTPs for local accounts.	phone_number, otpCode

2.2 Detailed API Request / Response Formats

1. Large Language Model (GenAI) API

Purpose: Invoked during Doubt Resolution and Quiz Generation loops to deliver contextual text outputs.

Sample Request Payload (JSON):

```
{
  "task_type": "DOUBT_RESOLUTION",
  "studentLanguage": "hi",
  "prompt_text": "What is Newton's Second Law of Motion?"
}
```

Sample Expected Response (JSON):

```
{
  "generated_answer_text": "न्यूटन के गति के दूसरे नियम के अनुसार, किसी वस्तु के संवेग परिवर्तन की दर...",
  "confidenceScore": 0.95,
  "fallbackToEnglish": false
}
```

2. Google OAuth 2.0 Identity API

Purpose: Validates the unique federated token sent by the mobile client app upon tapping "Continue with Google".

Sample Request Payload (JSON):

```
{
  "grant_type": "authorization_code",
  "code": "auth_code_from_mobile_client",
  "client_id": "vidyasaathi_client_id"
}
```

Sample Expected Response (JSON):

```
{
  "sub": "unique_google_user_id_108234",
  "email": "student@gmail.com",
  "email_verified": true,
  "name": "Student Name"
}
```

3. SMS Gateway API

Purpose: Handles the 6-digit verification codes required to transition local accounts from inactive to active state.

Sample Request Payload (JSON):

```
{  
  "phoneNumber": "+91-9000000000",  
  "otpCode": "482913"  
}
```

Sample Expected Response (JSON):

```
{  
  "verified": true,  
  "attemptsRemaining": 2,  
  "errorCode": null  
}
```

2.3 Critical API Exception & Edge Case Handling Rules

To ensure a high-quality codebase, three clear operational safety boundaries are enforced:

1. **AI Timeout or Low Confidence Fallback:** If the external LLM call takes longer than 5 seconds or yields a low confidence score.
2. **OAuth Account Duplication Safety:** If a student signs up using Google OAuth, the system verifies the incoming email against the database before provisioning a new row.
3. **Application-Side OTP Expiry Enforcement:** Independent of the gateway's status trackers, the system enforces a strict **10-minute TTL (Time-To-Live)** timestamp within the application domain.